```
}

function c() {
  ❻ let spam = "Coyote";
    document.write("spam is " + spam + "<br />");
}

❼ a();
</script>
```

When you run this code, the output looks like this:

```
spam is Ant
spam is Bobcat
spam is Coyote
spam is Bobcat
spam is Ant
```

When the program calls function `a()` ❼, a frame object is created and placed on the top of the call stack. This frame stores any arguments passed to `a()` (in this case, there are none), along with the local variable `spam` ❶ and the place where the execution should go when the `a()` function returns.

When `a()` is called, it displays the contents of its local `spam` variable, which is `Ant` ❷. When the code in `a()` calls function `b()` ❸, a new frame object is created and placed on the call stack above the frame object for `a()`. The `b()` function has its own local `spam` variable ❹, and calls `c()` ❺. A new frame object for the `c()` call is created and placed on the call stack, and it contains `c()`'s local `spam` variable ❻. As these functions return, the frame objects pop off the call stack. The program execution knows where to return to, because that return information is stored in the frame object. When the execution has returned from all function calls, the call stack is empty.

Figure 1-7 shows the state of the call stack as each function is called and returns. Notice that all the local variables have the same name: `spam`. I did this to highlight the fact that local variables are always separate variables with distinct values, even if they have the same name as other local variables.

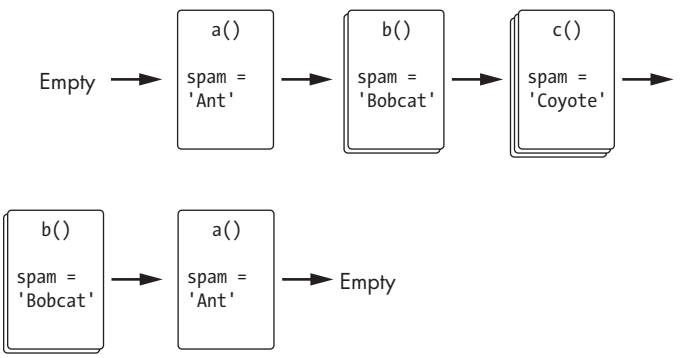

Figure 1-7: The state of the call stack as the localVariables *program runs*